

PROVA G1 – TIPO 4

Gabarito

Questão Resp.

01	B
02	C
03	B
04	A
05	B
06	C
07	C
08	B
09	A
10	C
11	B
12	B
13	B
14	C
15	B
16	DIP

1 – B.

I, II e III estão corretas. Encapsulamento não exige atributos públicos; na realidade, frequentemente busca justamente restringir o acesso direto.

2 – C.

Descrever como o programa deve executar → imperativo.

Descrever propriedades/relações e deixar o mecanismo determinar parte da execução → declarativo.

3 – B.

Funções armazenadas em variáveis, passadas como argumentos e retornadas por outras funções são **funções de primeira classe**.

4 – A.

Fatos + regras + consultas → inferência lógica.

5 – B.

I, II e IV estão corretas. A III é falsa: utilizar múltiplos paradigmas não significa que todos devam ser utilizados em todos os módulos.

6 – C.

Concentrar uma lógica repetida em uma única implementação → **DRY**.



7 – C.

Evitar complexidade desnecessária → **KISS** (*Keep It Simple, Stupid*).

8 – B.

I = alta coesão.

IV = baixo acoplamento.

II e III são características indesejáveis.

9 – A.

Um método/classe acumulando validação, cálculo financeiro e notificação possui responsabilidades distintas. A abordagem é separar responsabilidades → **SRP**.

10 – C.

I, II e III estão corretas. O OCP não significa que uma classe seja literalmente proibida de ser modificada; significa projetá-la para que novas funcionalidades possam ser adicionadas por extensão, reduzindo alterações no código existente.

11 – B.

Microservices permitem desenvolvimento e implantação independentes dos serviços.

12 – B.

Database per Service permite que cada serviço escolha a tecnologia de persistência mais adequada ao seu domínio.

13 – B.

V1, V2, V3 representam versões sequenciais das alterações do banco.

14 – C.

Fornecer configurações durante a execução, fora do código → **Externalized Configuration**.

15 – B.

I, II e IV estão corretas. A centralização não exige que todos os serviços tenham exatamente os mesmos valores.

Questão 16 – Tipo 4 — DIP

A questão apresenta `ProcessadorPedido` criando diretamente `new MySQLRepositorio()`.

a) Resposta esperada

O princípio violado é o **DIP — Dependency Inversion Principle**.

A classe de alto nível: `ProcessadorPedido` depende diretamente de uma implementação concreta: `MySQLRepositorio`



Isso cria um forte acoplamento. Se a aplicação precisar utilizar PostgreSQL ou outra tecnologia de persistência, será necessário alterar `ProcessadorPedido`.

b) Solução

Criar uma abstração:

```
interface Repositorio {  
    void salvar(Pedido pedido);  
}
```

Implementação MySQL:

```
class MySQLRepositorio implements Repositorio {  
  
    @Override  
    public void salvar(Pedido pedido) {  
        // salvar no MySQL  
    }  
}
```

Outra implementação:

```
class PostgreSQLRepositorio implements Repositorio {  
  
    @Override  
    public void salvar(Pedido pedido) {  
        // salvar no PostgreSQL  
    }  
}
```

E `ProcessadorPedido` passa a receber a abstração:

```
class ProcessadorPedido {  
  
    private final Repositorio repositorio;  
  
    public ProcessadorPedido(Repositorio repositorio) {  
        this.repositorio = repositorio;  
    }  
  
    public void processar(Pedido pedido) {  
        repositorio.salvar(pedido);  
    }  
}
```

A composição poderia então ser:

```
Repositorio repositorio = new MySQLRepositorio();  
  
ProcessadorPedido processador =  
    new ProcessadorPedido(repositorio);
```

Ou:

```
Repositorio repositorio = new PostgreSQLRepositorio();
```



```
ProcessadorPedido processador =  
    new ProcessadorPedido(repositorio);
```

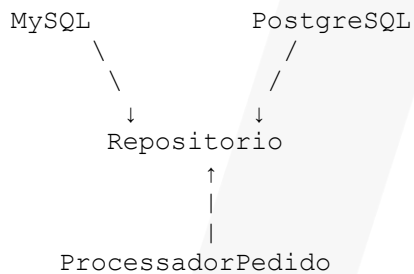
Explicação

Antes:

```
ProcessadorPedido  
    |  
    v  
MySQLRepositorio
```

O `ProcessadorPedido` conhece e cria diretamente a implementação.

Depois:



`ProcessadorPedido` conhece apenas: `Repositorio`

e não precisa saber se a implementação utiliza MySQL, PostgreSQL, MongoDB etc.

Esse é o conceito central do **DIP**:

**Módulos de alto nível não devem depender diretamente de módulos de baixo nível;
ambos devem depender de abstrações.**